

Unidade: Programação Orientada a Aspectos com Spring

Carga horária sugerida: 12 horas-aula

Foco: material conceitual para apresentação em sala

1. Problema que a AOP resolve

A Programação Orientada a Objetos (POO) organiza bem regras de negócio, mas não resolve de forma elegante funcionalidades que se repetem em vários lugares do sistema.

Exemplos clássicos:

- Logging
- Auditoria
- Monitoramento (tempo, contadores)
- Segurança
- Tratamento padronizado de exceções

Quando essas preocupações entram em vários métodos, surgem dois problemas:

- **Scattering (espalhamento):** o mesmo código aparece em várias classes.

2. Ideia central da AOP

A AOP (Aspect-Oriented Programming) separa os **interesses transversais** da lógica principal.

Sem AOP:

- ProdutoService = regra de negócio + log + monitoramento

Com AOP:

- ProdutoService = regra de negócio
- ObservabilidadeAspect = log + tempo + status

Resultado esperado:

- Código de negócio mais limpo
- Menos repetição

3. Conceitos fundamentais

3.1 Aspect

Classe que encapsula comportamento transversal.

3.2 Join Point

Ponto de execução onde o aspecto pode atuar (no Spring AOP, tipicamente execução de método de bean Spring).

3.3 Pointcut

Expressão que seleciona quais join points serão interceptados.

Exemplo de ideia:

- Todos os métodos de classes em `service`
- Todos os métodos de classes em `controller`

4. Spring AOP na pratica

4.1 O que o Spring oferece

No Spring, você cria aspectos com anotações, sem precisar de configuração complexa.

Dependencia Maven:

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-aspectj</artifactId>  
</dependency>
```

4.2 Como o Spring aplica o aspecto

- O Spring cria proxies para beans gerenciados (`@Service` , `@Component` , `@Repository` , `@RestController`).
- Ao chamar o bean pelo container, o proxy executa o advice antes/depois do

5. Limites importantes (muito cobrado em prova)

Spring AOP:

- Atua em beans gerenciados pelo container Spring.
- Intercepta execução de métodos (modelo proxy-based).

Não cobre diretamente:

- Objetos criados com `new` fora do container
- Construtores
- Campos
- Métodos `private`

Observação importante:

- **Self-invocation:** chamada de um método para outro dentro da mesma classe normalmente não passa pelo proxy, então o advice pode não disparar

6. Quando usar e quando evitar

Use AOP para:

- Logging técnico
- Auditoria
- Métricas e tempo de execução
- Políticas transversais de segurança

Evite AOP para:

- Regra de negócio principal
- Cálculo de domínio
- Decisões centrais de caso de uso

Regra pratica:

Se for transversal, use AOP. Se não, não use.

7. Roteiro conceitual para apresentação (sugestão 35-45 min)

1. Problema real: código repetido (5 min)
2. Conceitos: Aspect, Join Point, Pointcut, Advice (10 min)
3. Spring AOP e proxies (8 min)
4. Limitações e armadilhas (7 min)
5. Exemplo no projeto da turma (10-15 min)

8. Exemplo prático no backend deste projeto

8.1 O que foi adicionado

1. Dependência AOP no Maven:

- arquivo: `backend/produtos-api/pom.xml`

2. Aspecto de observabilidade:

- arquivo: `backend/produtos-api/src/main/java/br/unifor/produtosapi/aspect/ObservabilidadeAspect.java`
- intercepta métodos de `controller` e `service`
- registra início/fim, tempo em ms e status de sucesso/erro

8.2 Código do aspecto (resumo)

```
@Aspect  
@Component
```

9. Perguntas de fixação

1. O que caracteriza um interesse transversal?
2. Diferencie scattering e tangling.
3. Qual a diferença entre pointcut e advice?
4. Por que `@Around` é tão usado em observabilidade?
5. Por que self-invocation pode impedir a interceptação?
6. Quando você não deveria usar AOP?

10. Fechamento

A AOP com Spring não substitui modelagem de domínio: ela complementa a arquitetura separando preocupações transversais. Em projetos reais, essa separação reduz duplicação, melhora manutenção e facilita evolução do sistema.